

Assertions for Free: Transferring Invariants from Algorithm to Implementation Proofs (Functional Pearl)

SHUSHU WU, Shanghai Jiao Tong University, China

CHENGXI YANG, Shanghai Jiao Tong University, China

XIWEI WU, Shanghai Jiao Tong University, China

QINXIANG CAO, Shanghai Jiao Tong University, China

Verifying the functional correctness of real-world code with complex algorithms can be decomposed into two layers: verifying that the concrete code refines an abstract algorithmic description, and proving the correctness of the formal description. However, in practice the two layers do not stay cleanly separated. For example, in the verification of the Knuth-Morris-Pratt (KMP) algorithm, the implementation correctness proof often re-establishes algorithm properties that have already been proved, as the concrete implementation relies on invariants that the traditional two-layer method provides no mechanism to transfer. This makes it difficult to clearly separate the concerns of algorithm correctness and implementation correctness. In this pearl, we show how a clean separation can be achieved within the two-layer method by combining two simple ideas: expressing implementation correctness as a relational Hoare quadruple, and introducing assertion annotations into the abstract program to capture key invariants. Properties established in the algorithm proof are thereby transferred directly to the implementation proof, eliminating the need to re-prove them. We demonstrate the effectiveness of this approach through non-trivial case studies, including the Knuth-Morris-Pratt pattern-matching algorithm and the depth-first search algorithm, showing that it leads to simpler proofs and a more modular verification process.

CCS Concepts: • **Theory of computation** → **Logic and verification; Hoare logic; Invariants**; • **Software and its engineering** → Software verification.

Additional Key Words and Phrases: relational Hoare logic, assertion annotations, program refinement, two-layer verification, mechanized verification

ACM Reference Format:

Shushu Wu, Chengxi Yang, Xiwei Wu, and Qinxiong Cao. 2026. Assertions for Free: Transferring Invariants from Algorithm to Implementation Proofs (Functional Pearl). *Proc. ACM Program. Lang.* 10, ICFP, Article 296 (August 2026), 23 pages. <https://doi.org/10.1145/3828694>

1 Introduction

When verifying a program that implements a known algorithm, there is a natural temptation to reuse the algorithm's correctness proof. A commonly adopted strategy is the two-layer method [2], which separates proofs into: (1) implementation correctness, which ensures that the implementation program accurately implements an abstract algorithm description, and (2) algorithm correctness, which validates the abstract description's correctness. For example, consider the insertion function for binary search trees. The following triple can describe the functional correctness [11], where

Authors' Contact Information: Shushu Wu, Shanghai Jiao Tong University, Shanghai, China, Ciel77@sjtu.edu.cn; Chengxi Yang, Shanghai Jiao Tong University, Shanghai, China, yangarcadia@gmail.com; Xiwei Wu, Shanghai Jiao Tong University, Shanghai, China, yashen@sjtu.edu.cn; Qinxiong Cao, Shanghai Jiao Tong University, Shanghai, China, caoxinxiang@gmail.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/8-ART296

<https://doi.org/10.1145/3828694>

$\text{storemap}(p_0, m)$ denotes that there exists an abstract data structure tree t_0 stored at address p_0 (i.e. $\text{storetree}(p_0, t_0)$), and the key-value pairs of this tree correspond to a mapping m (i.e. $t_0 \sim m$).

$$\{\text{storemap}(p_0, m)\} \text{ tree_ins}(k, v, p_0) \{\text{storemap}(p_0, m[k \mapsto v])\} \quad (1)$$

Following the two-layer method, we introduce a functional model insert in Rocq [3] and split the functional-correctness triple (1) into two obligations, implementation correctness (2), which relates tree_ins to insert , and algorithm correctness (3), which establishes the properties of insert :

$$\{\text{storetree}(p_0, t_0)\} \text{ tree_ins}(k, v, p_0) \{\text{storetree}(p_0, \text{insert}(k, v, t_0))\} \quad (2)$$

$$\forall t_0 k v m. \text{bst}(t_0) \wedge t_0 \sim m \Rightarrow \text{bst}(\text{insert}(k, v, t_0)) \wedge \text{insert}(k, v, t_0) \sim m[k \mapsto v] \quad (3)$$

Here, $\text{bst}(t)$ denotes that t satisfies the binary search tree property, i.e., for every node, all keys in its left subtree are smaller and all keys in its right subtree are larger than the node's key. Such a two-layered proof approach offers substantial advantages, particularly in terms of modularity and reusability of proofs. By separating the algorithm correctness from the concrete implementation, the correctness of the algorithm can be verified independently of the specific implementation details. This separation allows algorithm proofs to be reused across multiple implementations, promoting efficiency in verifying different versions of the same algorithm. Moreover, modifications to the code do not require re-proving the algorithm's correctness.

Yet in practice, this turns out to be more subtle than it appears. The algorithm correctness is typically assumed during implementation, and properties such as safety conditions or invariants are implicitly carried over. As a result, the implementation correctness proof often requires re-proving properties already established during the algorithm correctness proof, leading to redundant efforts.

To illustrate this challenge, consider the computation of the “next” array (partial match table) in the Knuth-Morris-Pratt (KMP) pattern-matching algorithm, shown in Fig. 1. In the abstract algorithm on the left, we must prove that the list l_{next} computed from the pattern string l_{str} is correct. Specifically, $\text{nth } j \ l_{\text{next}}$ computes the j th element of the list l_{next} and returns a default value if j exceeds the length of l_{next} . Then, $\text{nth } j \ l_{\text{next}}$ should represent the length of the longest proper prefix of $l_{\text{str}}[0..j]$ that is also a suffix of $l_{\text{str}}[0..j]$. During this proof, we need to establish an important invariant: the index j used for prefix matching always remains less than the length of both the pattern l_{str} and the computed list l_{next} , ensuring that the list access $\text{nth } j \ l_{\text{str}}$ and $\text{nth } j \ l_{\text{next}}$ are valid and do not return the default value. However, in the concrete C implementation, as shown on the right side of Fig. 1, the programmer typically does not include explicit checks for $\text{next}[q]$, assuming that the algorithm itself ensures the correctness of the array bounds. This assumption, while reasonable from a programming perspective, creates a challenge for verification: the implementation correctness proof must establish memory safety without access to the algorithm's invariants. Concretely, under the traditional two-layer method, the abstract algorithm is formalized as a total function $\text{GetNext}^A : \text{list } \mathbb{Z} \rightarrow \text{list } \mathbb{Z}$, and implementation correctness reduces to proving the following Hoare triple:

$$\begin{aligned} & \{m = \text{len } l \wedge \text{storearray}(\text{str}, l)\} \\ & \text{GetNext}(\text{str}, m) \\ & \{r. \text{storearray}(\text{str}, l) * \text{storearray}(r, \text{GetNext}^A(l))\} \end{aligned}$$

Here, $\text{storearray}(\text{str}, l)$ denotes that the array str stores the list l . In the postcondition, the binder r names the return value, and $*$ is the separating conjunction of separation logic [24], asserting that str and r occupy disjoint regions of memory. The postcondition thus states that r points to an array storing $\text{GetNext}^A(l)$, the correct “next” array for l . To prove this triple, establishing that $q < m$ at every loop iteration is essential for memory safety. The bound $q < m$ is not a coincidence: it is a consequence of how the KMP algorithm works, and the algorithm correctness proof establishes

Definition $\text{GetNext}^A l_{\text{str}} :=$ <pre> $l_{\text{next}} := \{ 0 \}; i := 1; j := 0;$ while ($i < \text{len } l_{\text{str}}$) { while ($j > 0$ and $(\text{nth } j \ l_{\text{str}} \neq \text{nth } i \ l_{\text{str}})$) $j := \text{nth } (j - 1) \ l_{\text{next}};$ if ($\text{nth } j \ l_{\text{str}} = \text{nth } i \ l_{\text{str}}$) then $j := j + 1;$ $l_{\text{next}} := l_{\text{next}} \uparrow\uparrow \{ j \}; i := i + 1;$ return l_{next} </pre>	<pre> int * GetNext(int * str,int m) { int *next = (int *)calloc(m, sizeof(int)); next[0] = 0; int p = 1; int q = 0; while (p < m) { while (q > 0 && str[q] != str[p]) q = next[q - 1]; if (str[q] == str[p]) q = q + 1; next[p] = q; p = p + 1; } return next; } </pre>
---	--

Fig. 1. Computing the KMP “next” array: for each position i in the pattern string l_{str} , $l_{\text{next}}[i]$ stores the length of the longest proper prefix of $l_{\text{str}}[0..i]$ that is also a suffix of $l_{\text{str}}[0..i]$. For example, given pattern “abab”, the result is $[0, 0, 1, 2]$ since “abab” has “ab” (length 2) as both a proper prefix and suffix. Left: abstract algorithm operating on lists. Right: C implementation operating on arrays, where p and q correspond to i and j in the abstract algorithm.

it carefully as an invariant of j . Yet the implementation proof cannot see this. The two proofs live in separate worlds, and a fact established in one does not automatically carry into the other. The traditional method provides no mechanism to transfer it, so the implementation proof must re-establish the bound from scratch, duplicating work that has already been done.

The trick, it turns out, is to annotate the abstract program with assertions that fail in exactly the situations where the implementation would be unsafe. Once the algorithm proof establishes that these assertions never fail, the implementation proof receives the asserted properties for free, with no duplication of effort. In this pearl, we show how to realize this idea by expressing implementation correctness as a relational Hoare quadruple, whose error simulation condition is what makes the transfer work.

Concretely, we replace the standard Hoare triple for implementation correctness with a relational Hoare quadruple [7]. A relational Hoare quadruple $\{\mathcal{P}\} c^I \lesssim c^A \{Q\}$ states that if the initial states (σ_1^I, σ_1^A) satisfy the precondition $\mathcal{P}$, then for any execution of the implementation program c^I , either the abstract program triggers an error, or there exists an execution of the abstract program c^A that simulates the execution of c^I , with their final states (σ_2^I, σ_2^A) satisfying the postcondition Q . In addition, if the implementation program encounters an error from the initial state σ_1^I , the abstract program must also trigger an error. This error simulation condition is precisely what makes the assertion annotations work.

An assertion $\text{assert}(R^A)$ inserted into the abstract program succeeds silently when R^A holds and triggers an error otherwise. Combined with the quadruple, this gives the one-sided ASSERT rule:

$$\text{ASSERT} \frac{\{\mathcal{P} \wedge R^A\} c^I \lesssim c^A \{\Phi\}}{\{\mathcal{P}\} c^I \lesssim \text{bind } \text{assert}(R^A) c^A \{\Phi\}}$$

which says that once the abstract program is about to assert R^A , the verifier of the implementation may strengthen the precondition with R^A . In the KMP example, inserting $\text{assert}(j < \text{len } l_{\text{str}})$ before the inner loop makes the abstract program error whenever the index would be out of bounds. The algorithm correctness proof establishes that this assertion never fails on valid inputs; the ASSERT rule then makes $j < \text{len } l_{\text{str}}$ available to the implementation proof as a free fact, exactly the bound needed to justify the C access $\text{str}[q]$, with no separate proof required. Our main contributions are as follows:

- We present a methodology for transferring algorithm-level invariants directly into the implementation correctness proof. Specifically, relational Hoare quadruples and assertion annotations together ensure that properties established in the algorithm proof transfer directly to the implementation proof, with no duplication of effort.
- We demonstrate the practical effectiveness of our method through two non-trivial case studies, each exhibiting a distinct payoff of the methodology: the Knuth-Morris-Pratt (KMP) next-array computation gives a clean *separation of concerns* between algorithm correctness and implementation safety, and a Depth-First Search (DFS) algorithm shows *reuse across implementations*.
- Our case studies are mechanized in Rocq [30].

Outline. Sec. 2 introduces the two technical foundations: state relation monads with error-exit mechanisms, and relational Hoare quadruples. Sec. 3 develops the method using KMP as a running example. Sec. 4 applies the same ideas to DFS, where a single abstract algorithm with nondeterministic choice models two concrete implementations at once. Sec. 5 discusses when the approach works well, and compares it with an alternative approach to the KMP verification. Sec. 6 reviews related work and Sec. 7 concludes.

2 Background: Programs, Assertions, and Refinement

This section develops the two technical foundations of our approach: the language in which abstract algorithms are written, and the logic used to relate them to concrete implementations. The first (Sec. 2.1) centers on the `assert`(P^A) operation, which is the mechanism by which invariants are transferred between the two proof layers. The second (Sec. 2.2) introduces relational Hoare quadruples, which formalize what it means for a C program to refine an abstract specification.

2.1 State Relation Monads with Error Exit Mechanism

In our approach, the abstract algorithm is annotated with assertion statements that mark points where invariants hold. The operation that makes this possible is `assert`(P^A): when the abstract program asserts a property P^A , that property becomes available to the implementation proof. To give `assert`(P^A) a precise semantics, we model abstract algorithms as programs in a state monad that supports both normal termination and error exit. A monad [22] consists of a type constructor M along with two operations. Let A and B denote type variables:

- `return`: $A \rightarrow M A$, which lifts a pure value of type A into a computation of type $M A$.
- `bind`: $M A \rightarrow (A \rightarrow M B) \rightarrow M B$, which sequences a computation producing a value of type A with a continuation that takes a value of type A and returns a computation of type $M B$, yielding a computation of type $M B$.

Let Σ^A denote the type signature of the abstract state space, and let R denote the return type. The state relation monad with an error exit mechanism $\text{St}^E R$ consists of two components. For a computation $c^A \in \text{St}^E R$ (a program in this monad), the evaluation $\llbracket c^A \rrbracket_{\text{norm}} \subseteq \Sigma^A \times R \times \Sigma^A$ collects all possible state transitions, and $\llbracket c^A \rrbracket_{\text{err}} \subseteq \Sigma^A$ collects all states from which the computation triggers an error:

- $(\sigma_1^A, r, \sigma_2^A) \in \llbracket c^A \rrbracket_{\text{norm}}$: from state σ_1^A the program c^A may terminate at state σ_2^A and return value r ;
- $\sigma_1^A \in \llbracket c^A \rrbracket_{\text{err}}$: from state σ_1^A the program c^A results in an error.

The return operation `return` r always returns the value r without modifying the state and never results in an error. The bind operation `bind` $c^A f^A$ sequences two computations: first execute c^A , and if it succeeds with value r_0 in state σ_0^A , then execute $f^A(r_0)$ starting from state σ_0^A . The

bind operation may result in an error in two cases: either c^A itself fails, or c^A succeeds but the continuation f^A fails. Formally:

$$\begin{aligned} (\sigma_1^A, r, \sigma_2^A) \in \llbracket \text{bind } c^A f^A \rrbracket_{\text{nrm}} & \quad \text{iff. } \exists \sigma_0^A r_0 \text{ s.t. } (\sigma_1^A, r_0, \sigma_0^A) \in \llbracket c^A \rrbracket_{\text{nrm}} \text{ and} \\ & \quad (\sigma_0^A, r, \sigma_2^A) \in \llbracket f^A(r_0) \rrbracket_{\text{nrm}} \\ \sigma_1^A \in \llbracket \text{bind } c^A f^A \rrbracket_{\text{err}} & \quad \text{iff. } \sigma_1^A \in \llbracket c^A \rrbracket_{\text{err}} \text{ or } \exists \sigma_0^A r_0 \text{ s.t.} \\ & \quad (\sigma_1^A, r_0, \sigma_0^A) \in \llbracket c^A \rrbracket_{\text{nrm}} \text{ and } \sigma_0^A \in \llbracket f^A(r_0) \rrbracket_{\text{err}} \end{aligned}$$

The key operation is $\text{assert}(P^A) : \text{St}^E$ unit: it succeeds silently when P holds in the current state, and exits with an error otherwise. This error exit is what connects the two proof layers: the algorithm correctness proof establishes that P^A always holds at the assertion point, so $\text{assert}(P^A)$ never triggers an error. The implementation proof can therefore use P^A as a guaranteed fact at that point, without having to prove it independently. The semantics of $\text{assert}(P^A)$ is defined as follows, where $()$ is the only value of type unit (since the operation produces no meaningful return value):

$$\begin{aligned} (\sigma_1^A, r, \sigma_2^A) \in \llbracket \text{assert}(P^A) \rrbracket_{\text{nrm}} & \quad \text{iff. } \sigma_1^A = \sigma_2^A, \sigma_1^A \in P^A \text{ and } r = () \\ \sigma^A \in \llbracket \text{assert}(P^A) \rrbracket_{\text{err}} & \quad \text{iff. } \sigma^A \notin P^A \end{aligned}$$

Additionally, to model while loops and recursion, we use a fixed-point operator fix , which is based on Kleene's fixed-point theorem. Details on the implementation are provided in Appendix A. With these monadic operations (return, bind, assert, and fix), we can formalize the abstract algorithm for computing the “next” array in the KMP algorithm.

2.2 Relational Hoare Quadruples with Error

To formalize that a C implementation faithfully implements an abstract algorithm, we use relational Hoare logic [7, 31]. A relational Hoare quadruple of the form

$$\{\mathcal{P}\} c_1 \preceq c_2 \{r_1, r_2 \cdot \Phi \ r_1 \ r_2\}$$

states that if the initial states satisfy the precondition $\mathcal{P}$, then:

- *result simulation*: For any execution of program c_1 with return value r_1 , either program c_2 triggers an error, or there exists an execution of c_2 that returns value r_2 , with the final states satisfying the postcondition $\Phi \ r_1 \ r_2$.
- *error simulation*: If program c_1 exits with an error, then program c_2 must also trigger an error.

When the precondition $\mathcal{P}$ and postcondition Φ establish appropriate relationships between the states and return values, the relational quadruple signifies that program c_1 refines program c_2 . The error simulation condition is a general strength of relational Hoare logic: if the right-hand program never triggers an error on valid inputs, the quadruple guarantees that the left-hand program also never encounters one. In verification of C programs against abstract specifications, this means memory safety follows directly from the algorithm's correctness, without a separate proof. The same principle applies in other relational settings, such as compiler correctness, where it ensures that a source program's safety is preserved by compilation. In our setting, this condition also makes assertion annotations work: the algorithm correctness proof establishes that $\text{assert}(P^A)$ never triggers an error on valid inputs, and the quadruple then guarantees that the implementation never reaches a state where P^A fails, so P^A is available to the implementation proof as a fact.

In our application, we instantiate this framework to relate a C implementation program c^I with an abstract program c^A written in the state relation monad with error exit mechanism defined in the previous subsection. The two programs live in different semantic domains: c^I operates over a concrete state space Σ^I , while c^A operates over an abstract state space Σ^A . Accordingly, the precondition $\mathcal{P} \subseteq \Sigma^I \times \Sigma^A$ is a relation on paired concrete and abstract initial states (σ_1^I, σ_1^A) . The postcondition $\Phi \subseteq V_I \times V_A \times \Sigma^I \times \Sigma^A$ relates the concrete return value r^I , the abstract return value

$$\begin{array}{c}
\text{FOCUS-LEFT} \frac{\begin{array}{c} \{\mathcal{P}\} c_1^I \lesssim \text{return } () \{\mathcal{R}\} \\ \{\mathcal{R}\} c_2^I \lesssim c^A \{\Phi\} \end{array}}{\{\mathcal{P}\} c_1^I; c_2^I \lesssim c^A \{\Phi\}} \quad \text{FOCUS-RIGHT} \frac{\begin{array}{c} \{\mathcal{P}\} \text{skip} \lesssim c^A \{r^A \cdot \Psi \ r^A\} \\ \forall v. \{\Psi \ v\} c^I \lesssim f^A v \{\Phi\} \end{array}}{\{\mathcal{P}\} c^I \lesssim \text{bind } c^A f^A \{\Phi\}} \\
\\
\text{WHILE-SIM} \frac{\forall u. \{\Psi \ u \wedge b^I \wedge b^A\} c^I \lesssim c^A \{r^A \cdot \Psi \ r^A\} \quad \forall w. \Psi \ w \Rightarrow (b^I \Leftrightarrow b^A)}{\{\Psi \ v\} \text{while } b^I \text{ do } c^I \lesssim \text{while } b^A \text{ do } c^A \{r^A \cdot \Psi \ r^A \wedge \neg b^I \wedge \neg b^A\}} \\
\\
\text{RET} \frac{}{\{\Phi \ u \ v\} \text{return } u \lesssim \text{return } v \{\Phi\}} \quad \text{ASSERT} \frac{\{\mathcal{P} \wedge R^A\} c^I \lesssim c^A \{\Phi\}}{\{\mathcal{P}\} c^I \lesssim \text{bind } \text{assert}(R^A) c^A \{\Phi\}}
\end{array}$$

Fig. 2. An excerpt of the relational inference rules.

r^A , and their respective final states, where V_I and V_A denote the concrete and abstract return-value types. The error behavior of c^A is given by $\llbracket c^A \rrbracket_{\text{err}}$ as defined in the previous subsection, while errors in the C program c^I correspond to undefined behaviors such as out-of-bounds array accesses or null pointer dereferences. A quadruple of the form

$$\{\mathcal{P}\} c^I \lesssim c^A \{r^I, r^A \cdot \Phi \ r^I r^A\}$$

then verifies that the C implementation c^I refines the abstract specification c^A across these two semantic domains.

Relational Hoare logic provides a rich set of inference rules for proving such a relational quadruple. These rules apply to various program structures, such as sequences, conditionals, and loops. We will present the core inference rules in the next section, illustrated using the KMP case study.

3 Case Study: Leveraging Algorithm Correctness for Safety Checks

We illustrate our approach using the computation of the KMP “next” array as a running example. The proof unfolds in three steps: expressing implementation correctness as a relational quadruple (Sec. 3.1), annotating the abstract algorithm with assertions to transfer the bound invariant (Sec. 3.2), and composing the two proofs to obtain functional correctness (Sec. 3.3).

3.1 Implementation Correctness as Relational Quadruples

For the KMP algorithm, the following quadruple captures the implementation for the computation of the “next” array:

$$\begin{array}{l}
\{m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}})\} \\
\text{GetNext}(\text{str}, m) \lesssim \text{GetNext}^A(l_{\text{str}}) \\
\{r^I r^A \cdot \text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(r^I, r^A)\}
\end{array} \tag{4}$$

This quadruple states that, if before execution, the pattern string variable str stores the abstract string l_{str} , then the implementation program refines the abstract algorithm, ensuring that their return values match.

To prove this quadruple, we need relational inference rules. Relational Hoare logic provides a rich set of inference rules that allow us to reason about different program structures, such as sequences, conditionals, and loops. Fig. 2 displays part of the core inference rules¹. Rule **FOCUS-LEFT** and **FOCUS-RIGHT** allow us to focus on the execution of implementation and abstract programs

¹A set of inference rules is provided in Appendix B.

individually, stepping through one side while leaving the other unchanged. When one side has no meaningful return value (e.g., the unit value $()$), we omit it from the postcondition.

For example, we apply rule **FOCUS-LEFT** to handle the C-side array allocation $\text{next} := \text{calloc}(m)$. This reduces the goal to two subgoals, where m Zeros denotes an abstract list of length m consisting of zeros:

$$\begin{aligned} &\{m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}})\} \\ &\quad \text{next} = (\text{int} *)\text{calloc}(m, \text{sizeof}(\text{int})) \lesssim \text{return } () \end{aligned} \quad (5)$$

$$\begin{aligned} &\{m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(\text{next}, m \text{ Zeros})\} \\ &\{m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(\text{next}, m \text{ Zeros})\} \\ &\quad \text{next}[0] = 0; i = 1; j = 0; \dots \lesssim \text{GetNext}^A(l_{\text{str}}) \end{aligned} \quad (6)$$

$$\{r^I r^A. \text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(r^I, r^A)\}$$

Subgoal (5) is a relational quadruple where the abstract side does nothing ($\text{return } ()$), and follows directly from the semantics of calloc . We then continue proving subgoal (6). Symmetrically, we apply rule **FOCUS-RIGHT** to evaluate the abstract-side initialization $l_{\text{next}} := \{0\}$. In **FOCUS-RIGHT**, the implementation side in the first premise is skip (doing nothing), so evaluating the abstract step does not change the C program. Applying **FOCUS-LEFT** and **FOCUS-RIGHT** similarly to the remaining initializations ($\text{next}[0] = 0; i = 1; j = 0$; on the C side and $i := 1, j := 0$ on the abstract side) yields the following precondition for the outer loop:

$$\begin{aligned} &m = \text{len } l_{\text{str}} \wedge p = i \wedge q = j \wedge \text{storearray}(\text{str}, l_{\text{str}}) * \\ &\quad \text{storearray}(\text{next}, l_{\text{next}}) * \text{arrayoffset}(\text{next}, (m - i) \text{ Zeros}, i) \end{aligned} \quad (7)$$

The $\text{arrayoffset}(x, l, n)$ expression indicates that the array l is stored starting at address $x + n$ (i.e., at address x offset by n). This assertion correlates the concrete array next with the abstract list l_{next} and tracks that positions 0 through $i - 1$ have been filled, while positions i through $m - 1$ remain as zeros. We want to use (7) as the loop invariant for the simultaneous while rule **WHILE-SIM**. However, this assertion does not provide information about the range of q , leaving the read access $\text{str}[q]$ potentially unsafe.

3.2 Inserting Assertion Annotations into the Abstract Algorithm

Rule **ASSERT** shows that when the abstract program contains $\text{assert}(R^A)$, the precondition $\mathcal{P}$ is strengthened to $\mathcal{P} \wedge R^A$ for the continuation. This means the implementation correctness proof may freely use R^A as an additional fact whenever the abstract program asserts it. These assertions are later discharged by the algorithm correctness proof, which establishes that the abstract program never triggers an assertion failure on valid inputs.

To be precise, we return to the outer loop invariant (7). Because this invariant does not constrain q , the C access $\text{str}[q]$ may be out of bounds. To handle this, we need the abstract program to also trigger an error whenever $j \geq \text{len } l_{\text{str}}$, so that the error simulation condition is satisfied. This is exactly what $\text{assert}(j < \text{len } l_{\text{str}})$ achieves: it makes the abstract program error precisely when the C program would. We therefore annotate the abstract program as shown in Fig. 3: before the inner while loop, we insert $\text{assert}(j < \text{len } l_{\text{str}})$. Applying rule **ASSERT** to this assertion, we obtain the strengthened precondition for the inner loop:

$$\begin{aligned} &i < m \wedge m = \text{len } l_{\text{str}} \wedge p = i \wedge q = j \wedge j < \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}}) * \\ &\quad \text{storearray}(\text{next}, l_{\text{next}}) * \text{arrayoffset}(\text{next}, (m - i) \text{ Zeros}, i) \end{aligned} \quad (8)$$

Definition $\text{GetNext}^A l_{\text{str}} :=$
 $l_{\text{next}} := \{ 0 \}; i := 1; j := 0;$
while $(i < \text{len } l_{\text{str}}) \{$
 $\text{assert } (j < \text{len}(l_{\text{str}}));$
 while $(j > 0 \text{ and } (\text{nth } j \ l_{\text{str}} \neq \text{nth } i \ l_{\text{str}})) \{$
 $\text{assert } (j < \text{len}(l_{\text{next}}));$
 $j := \text{nth } (j - 1) \ l_{\text{next}};$
 $\text{assert } (j < \text{len}(l_{\text{str}})) \}$
 if $(\text{nth } j \ l_{\text{str}} = \text{nth } i \ l_{\text{str}})$ **then** $j := j + 1;$
 $l_{\text{next}} := l_{\text{next}} \uparrow\uparrow \{ j \}; i := i + 1;$
return l_{next}

Fig. 3. Inserting Assertion Annotations for KMP Algorithm

The condition $j < \text{len } l_{\text{str}}$ is directly derived from the assertion by rule **ASSERT**. Together with $q = j$ in the invariant, this establishes $q < \text{len } l_{\text{str}}$, making the access $\text{str}[q]$ safe in the C program. The validity of the assertion $j < \text{len } l_{\text{str}}$ itself is not required here; it will be established separately by the algorithm correctness proof.

The same pattern applies inside the inner loop body (see Fig. 3): for each potentially unsafe C access, a corresponding assertion is inserted in the abstract program. Before the assignment $j = \text{next}[j-1]$, the abstract program asserts $j < \text{len } l_{\text{next}}$, which by rule **ASSERT** establishes $q < \text{len } \text{next}$, justifying the safe access $\text{next}[j-1]$. After this assignment, $\text{assert}(j < \text{len } l_{\text{str}})$ is inserted to re-establish the bound on q needed to restore the inner loop invariant (8) at the next iteration.

3.3 Composing Implementation and Algorithm Correctness

The relational quadruple (4) establishes that the C implementation refines the abstract program, but it does not by itself yield functional correctness. To derive functional correctness, we compose it with the algorithm correctness proof. The algorithm correctness proof establishes the correctness of the abstract program independently, and in particular implies that all assertion annotations inserted in Sec. 3.2 hold.

For the KMP example, we use the following Hoare triple to formalize the correctness of the abstract program:

$$\{\text{True}\} \text{GetNext}^A(l_{\text{str}}) \{r^A. \text{prefixFunc } r^A l_{\text{str}} \wedge \text{len } r^A = \text{len } l_{\text{str}}\} \quad (9)$$

Here, the unary triple $\{P\} c \{r. \Phi r\}$ holds when, on every state satisfying P , the program c runs error-free and every result it returns satisfies Φ . The predicate $\text{prefixFunc } r^A l_{\text{str}}$ denotes that for each position j , the j th element of r^A is the length of the longest proper prefix of $l_{\text{str}}[0..j]$ that is also a suffix of $l_{\text{str}}[0..j]$. Taken together, the triple states that the abstract program is guaranteed to produce a complete result list which correctly captures the prefix-suffix matching table of the input l_{str} . Intuitively, at iteration i , the variable j records the length of the longest proper prefix-suffix of the already-processed portion $l_{\text{str}}[0..i-1]$. The algorithm extends this match by one when $\text{nth } i \ l_{\text{str}} = \text{nth } j \ l_{\text{str}}$, and otherwise falls back to a shorter candidate; throughout, the loop maintains $j \leq i < \text{len } l_{\text{str}}$, exactly the bound the inserted assertions check. By definition of j , the trailing j characters of $l_{\text{str}}[0..i-1]$ are equal to $l_{\text{str}}[0..j-1]$. A shorter prefix-suffix of $l_{\text{str}}[0..i-1]$ has its suffix-part lying inside this trailing block, which is $l_{\text{str}}[0..j-1]$, and is therefore itself a prefix-suffix

of $l_{\text{str}}[0..j-1]$. The longest such is $\text{nth } j-1 \ l_{\text{next}}$, already in the table. The algorithm therefore descends this chain, extending when a character matches and falling further when it does not.

We can then compose this triple with quadruple (4) to derive functional correctness, a step we refer to as *vertical composition*. Relational Hoare logic provides the following rule for this purpose:

$$\text{VCOMP} \frac{\{\mathcal{P}\} \ c^I \preceq c^A \ \{\Phi\} \quad \{P^A\} \ c^A \ \{\Psi\}}{\{\mathcal{P} \odot P^A\} \ c^I \ \{r^I. \exists r^A. (\Phi \ r^I \ r^A) \odot (\Psi \ r^A)\}}$$

Here, the operator $\odot$ concatenates a relational assertion $\mathcal{P} \subseteq \Sigma^I \times \Sigma^A$ with a unary abstract assertion $P^A \subseteq \Sigma^A$ into a concrete assertion $\mathcal{P} \odot P^A \subseteq \Sigma^I$:

$$\sigma^I \models \mathcal{P} \odot P^A \text{ iff. } \exists \sigma^A \text{ s.t. } (\sigma^I, \sigma^A) \models \mathcal{P} \text{ and } \sigma^A \models P^A.$$

Intuitively, $\mathcal{P} \odot P^A$ requires the existence of an abstract state that simultaneously satisfies the relational precondition $\mathcal{P}$ and the algorithm precondition P^A . In the VComp rule, this operator is used twice: in the precondition, $\mathcal{P} \odot P^A$ combines the relational precondition with the algorithm's precondition; in the postcondition, $(\Phi \ r^I \ r^A) \odot (\Psi \ r^A)$ witnesses an abstract return value r^A that satisfies both the refinement postcondition and the algorithm's postcondition. The combined effect is a unary triple over c^I : c^I runs error-free on $\mathcal{P} \odot P^A$, and every result it returns satisfies the postcondition above. Applying this rule to quadruple (4) and triple (9), the precondition $\mathcal{P} \odot \text{True}$ requires the existence of an abstract state such that $m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}})$ holds, which is trivial. The postcondition $(\Phi \ r^I \ r^A) \odot (\Psi \ r^A)$ illustrates the existence of an abstract return value r^A satisfying both $\text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(r^I, r^A)$ from Φ and $\text{prefixFunc } r^A \ l_{\text{str}} \wedge \text{len } r^A = \text{len } l_{\text{str}}$ from Ψ . This yields the functional correctness of the concrete implementation as follows:

$$\begin{aligned} & \{m = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}})\} \\ & \text{GetNext}(\text{str}, m) \\ & \{r^I. \exists l. \text{prefixFunc } l \ l_{\text{str}} \wedge \text{len } l = \text{len } l_{\text{str}} \wedge \text{storearray}(\text{str}, l_{\text{str}}) * \text{storearray}(r^I, l)\} \end{aligned}$$

This triple establishes that the C implementation correctly computes the “next” array, with the result stored at r^I satisfying the prefix-suffix matching property.

4 Case Study: Leveraging Algorithm Correctness for Functional Correctness

In real-world applications, Depth-First Search (DFS) is a versatile algorithm with different use cases. As shown in Fig. 4, it can be used to sum the values of all reachable vertices (left), or to search for a specific vertex with a target value (right). Though both procedures share the same DFS traversal structure, their control flows differ: the sum program always traverses the full graph, while the search program may terminate early once the target is found. Their correctness descriptions also differ accordingly.

Despite these differences, both are variations of the same DFS procedure and can be modeled by a single abstract DFS program. This section introduces how nondeterministic choice in the abstract program simulates the different control flows (Sec. 4.1), and how assertion annotations help ensure functional correctness of the concrete implementations (Sec. 4.2).

4.1 Modeling Variations with Nondeterminism

The two programs in Fig. 4 differ in one key aspect: their termination condition. `dfs_sum` always traverses the full graph, breaking out of the loop only when no unvisited neighbors remain. `dfs_find`, on the other hand, may terminate early as soon as a vertex with the target value is found. Naively, this would require two separate abstract algorithms, one for each termination behavior, duplicating the proof effort for the shared traversal logic.

<pre> void dfs_sum(vertex* x, int* acc) { x→visited = 1; *acc += x→value; edge* e = x→vedge; while (e != NULL) { vertex* y = e→etail; if (y→visited == 0) dfs_sum(y, acc); e = e→next; } </pre>	<pre> vertex* dfs_find(vertex* x, int tgt) { x→visited = 1; if (x→value == tgt) return x; edge* e = x→vedge; vertex* r = NULL; while (e != NULL) { vertex* y = e→etail; if (y→visited == 0) { r = dfs_find(y, tgt); if (r != NULL) break; } e = e→next; } return r; } </pre>
--	---

Fig. 4. Two DFS-based concrete implementations. Left: `dfs_sum` sums the values of all reachable vertices, always traversing the full graph. Right: `dfs_find` searches for a vertex with a target value, terminating early when found.

Both programs share the same traversal logic — the only freedom is in *when* to exit the loop. If the abstract program does not commit to a termination condition, it can model both behaviors at once. This is exactly what nondeterminism gives us: the operation $\mathbf{NDchoice}(c_1^A, c_2^A)$ allows the abstract program to take either branch, leaving the choice open. Its semantics is:

$$\begin{aligned}
 (\sigma_1^A, r, \sigma_2^A) &\in \llbracket \mathbf{NDchoice}(c_1^A, c_2^A) \rrbracket_{\text{nrm}} && \text{iff. } (\sigma_1^A, r, \sigma_2^A) \in \llbracket c_1^A \rrbracket_{\text{nrm}} \text{ or } (\sigma_1^A, r, \sigma_2^A) \in \llbracket c_2^A \rrbracket_{\text{nrm}} \\
 \sigma_1^A &\in \llbracket \mathbf{NDchoice}(c_1^A, c_2^A) \rrbracket_{\text{err}} && \text{iff. } \sigma_1^A \in \llbracket c_1^A \rrbracket_{\text{err}} \text{ or } \sigma_1^A \in \llbracket c_2^A \rrbracket_{\text{err}}
 \end{aligned}$$

Fig. 5 shows the resulting abstract DFS algorithm `dfs_rec` (ignore the assertion for now; it will be discussed in Sec. 4.2). It first marks the current vertex u as visited via `mark_visited( $u$ )`, updating the abstract visited set `vis`. It then enters a repeat loop whose body is a nondeterministic choice $\mathbf{NDchoice}(c_1^A, c_2^A)$ offering two branches². The left branch immediately breaks out of the loop with false, modeling early termination. The right branch checks whether the current vertex u has any unvisited neighbor w : if so, it recurses into w via `dfs_rec( $g, s, w$ )`; otherwise, it breaks with true, signaling that the full graph has been explored. Thus, `dfs_rec` returns false when it exits early and true when it completes the full traversal.

This technique of using nondeterminism to model varying termination conditions is not specific to DFS; Kohlen et al. [17] employ the same idea in the abstract specification of an Interval Iteration algorithm for Markov Decision Processes, where the abstract program models termination nondeterministically to accommodate both early termination and continued iteration.

The nondeterministic branch does not affect the correctness of `dfs_rec`: whenever the algorithm completes the full traversal (returning true), the visited set `vis` is exactly the set of vertices reachable from u , as stated by the following triple:

$$\{\text{vis} = \emptyset\} \text{dfs_rec}(g, u, u) \{r^A. r^A = \text{true} \Rightarrow (\forall v. u \xrightarrow{g} v \Leftrightarrow v \in \text{vis})\} \quad (10)$$

Here, $u \xrightarrow{g} v$ denotes that vertex v is reachable from u in the graph g .

When verifying that a concrete program implements `dfs_rec`, the prover must handle the nondeterministic choice in `dfs_rec`. In relational Hoare logic, this is resolved *angelically*: the prover selects whichever branch of $\mathbf{NDchoice}(c_1^A, c_2^A)$ matches the concrete program's control flow. For `dfs_sum`, which always completes the traversal, the right branch is always taken. For `dfs_find`,

²The `break` by construct is conceptually similar to the `break` statement in Rust, which can exit a loop early and return a value, such as `break Some(value)`.

```

Record  $\Sigma^A : \text{Type} := \{ \text{vis} : \mathcal{P}(V); \}
Definition dfs_rec (g: Graph V E) (s u: V) : StE bool :=
    mark_visited(u);
    assert ( $s \overset{g}{\rightsquigarrow} u$ );
    b  $\leftarrow$  Repeat {
        NDchoice
        { break by false }
        { if ( $\exists w. (u, w) \in E \wedge w \notin \text{vis}$ )
            then dfs_rec(g, s, w)
            else break by true }
    };
    return b$ 
```

Fig. 5. Abstract DFS algorithm. The nondeterministic choice NDchoice allows the algorithm to either terminate early (breaking with false) or continue traversal, eventually breaking with true when all reachable vertices have been visited.

which may terminate early, the left branch is taken when the target is found. This is captured by the following rules:

$$\text{ND-LEFT} \frac{\{\mathcal{P}\} c^I \preceq c_1^A \{\Phi\}}{\{\mathcal{P}\} c^I \preceq \text{NDchoice}(c_1^A, c_2^A) \{\Phi\}} \qquad \text{ND-RIGHT} \frac{\{\mathcal{P}\} c^I \preceq c_2^A \{\Phi\}}{\{\mathcal{P}\} c^I \preceq \text{NDchoice}(c_1^A, c_2^A) \{\Phi\}}$$

Rule **ND-LEFT** says that if c^I refines the left branch, it refines the whole choice; **ND-RIGHT** is the symmetric counterpart. Together, they allow the same dfs_rec to serve as the abstract algorithm for both dfs_sum and dfs_find. In the next subsection, we focus on dfs_find: its early termination makes it the more interesting case for illustrating how assertion annotations transfer invariants from the abstract algorithm to the implementation proof. The proof for dfs_sum is simpler since it always completes the full traversal, and has also been formalized in Rocq against the same abstract algorithm.

4.2 Verifying the Correctness of a DFS Vertex Finding Program

Our ultimate goal is to prove the following functional correctness triple for the concrete DFS program. Here, g is the graph, u is the starting vertex, v_{tgt} is the target value to search for, and $f_{\text{value}} : V \rightarrow \mathbb{Z}$ maps each vertex to its stored value. The predicate storegraph(g, p, f_{value}) describes the memory layout of the graph's adjacency table, the visited status of each vertex captured by a vertex subset $p : \mathcal{P}(V)$, and the value of each vertex captured by f_{value} . Concretely, for each vertex v , it asserts that the visited field of v in memory is set to 1 if and only if $v \in p$, so p directly reflects the visited flags in memory. It also asserts that the value field of v in memory equals $f_{\text{value}}(v)$, so f_{value} reflects the stored values of vertices in memory. The predicate $u \overset{g}{\rightsquigarrow} v$ states that vertex v is

reachable from u in g :

$$\begin{aligned}
 & \{u \in g.V \wedge x = u \wedge \text{tgt} = v_{\text{tgt}} \wedge \text{storegraph}(g, \emptyset, f_{\text{value}})\} \\
 & \text{dfs_find}(x, \text{tgt}) \\
 & \{r^I . \exists p_{\text{vis}}. (r^I = \text{nullptr} \wedge (\forall v. u \xrightarrow{g} v \Leftrightarrow v \in p_{\text{vis}}) \quad (\text{target not found}) \\
 & \quad \vee r^I \neq \text{nullptr} \wedge f_{\text{value}}(r^I) = v_{\text{tgt}} \wedge u \xrightarrow{g} r^I) \quad (\text{target found}) \\
 & \quad \wedge \text{storegraph}(g, p_{\text{vis}}, f_{\text{value}})\}
 \end{aligned} \tag{11}$$

In the target-not-found case, the program returns null and the visited set p_{vis} is exactly the set of all vertices reachable from u , meaning the DFS has fully explored the graph from the source. In the target-found case, the program returns a non-null vertex whose value equals v_{tgt} and which is reachable from u . In both cases, $\text{storegraph}(g, p_{\text{vis}}, f_{\text{value}})$ asserts that the visited flags in memory reflect the final visited set p_{vis} .

Implementation Correctness. To derive triple (11), we follow the two-layer approach: show that dfs_find faithfully implements dfs_rec . This is formalized by the following relational quadruple, which relates the concrete execution to the abstract one³:

$$\begin{aligned}
 & \{u \in g.V \wedge x = u \wedge \text{tgt} = v_{\text{tgt}} \wedge \text{vis} = \emptyset \wedge \text{storegraph}(g, \text{vis}, f_{\text{value}})\} \\
 & \text{dfs_find}(x, \text{tgt}) \preceq \text{dfs_rec}(g, u, u) \\
 & \{r^I, r^A . (r^I = \text{nullptr} \wedge r^A = \text{true} \quad (\text{target not found}) \\
 & \quad \vee r^I \neq \text{nullptr} \wedge r^A = \text{false} \wedge f_{\text{value}}(r^I) = v_{\text{tgt}} \wedge u \xrightarrow{g} r^I) \quad (\text{target found}) \\
 & \quad \wedge \text{storegraph}(g, \text{vis}, f_{\text{value}})\}
 \end{aligned} \tag{12}$$

The precondition $\text{vis} = \emptyset$ and $x = u$ reflect that both programs start from the same vertex with no vertices visited. In the target-not-found case, both programs complete the full traversal: the implementation returns null and the abstract algorithm returns true. In the target-found case, the implementation terminates early with a non-null vertex carrying the target value v_{tgt} and reachable from u , while the abstract algorithm returns false. In both cases, $\text{storegraph}(g, \text{vis}, f_{\text{value}})$ records that the visited flags in memory are consistent with the abstract visited set vis .

Assertion Annotation. To prove quadruple (12), the nondeterministic choice in dfs_rec must be resolved, as illustrated in Fig. 6. When the implementation continues the traversal, rule **ND-RIGHT** selects the recursive branch of **NDchoice**(c_1^A, c_2^A), and this case follows straightforwardly. The challenging case is early termination: when the implementation returns a non-null vertex, rule **ND-LEFT** selects the exit branch, and we must show that the returned vertex is reachable from the source u . The precondition of (12) only states that $x = u$ and $\text{vis} = \emptyset$; it does not directly provide reachability of vertices encountered during recursion. However, reachability of the current vertex is a well-known invariant of the DFS algorithm: whenever dfs_find is called recursively on a neighbor x , x is always reachable from the source u . This invariant is established during the algorithm correctness proof of dfs_rec , but without an assertion annotation, it is not visible to the implementation correctness proof. We therefore insert the assertion $\text{assert}(s \xrightarrow{g} u)$ into the abstract algorithm right after $\text{mark_visited}(u)$, as shown in Fig. 5. By rule **ASSERT**, whenever the abstract program asserts $s \xrightarrow{g} u$, this fact is made available to the implementation correctness proof as part of the precondition for the continuation. Thus, for any vertex the implementation may return

³The proof for this example involves addressing recursive calls, which is not central to this pearl. These technical details are provided in Appendix C.

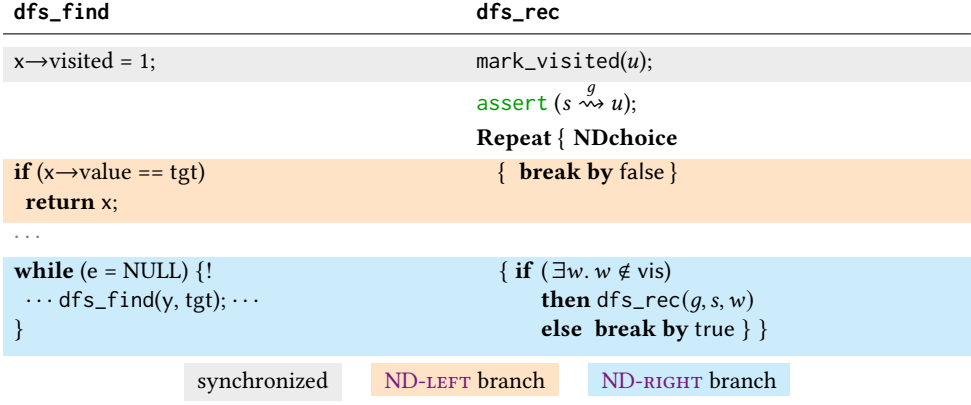


Fig. 6. Control flow matching between `dfs_find` and `dfs_rec`. The gray region marks the synchronized step. The orange region is matched against the left branch of NDchoice via rule **ND-LEFT**; the blue region is matched against the right branch via rule **ND-RIGHT**.

at this point, reachability from u is already established, and the correctness of early termination follows directly from the abstract algorithm's invariant. At a high level, the proof proceeds by synchronizing the two programs using the Focus rules, maintaining $\text{storegraph}(g, \text{vis}, f_{\text{value}})$ as the relational invariant through each recursive call, which directly captures the correspondence between the abstract visited set vis and the concrete visited flags in memory. A detailed proof of quadruple (12) is included in the artifact.

Vertical Composition. With quadruple (12) established, we apply the vertical composition rule **VComp** to compose it with triple (10), deriving the functional correctness triple (11). Applying rule **VComp** to (12) and (10) directly yields the following triple:

$$\begin{aligned}
 & \{ \exists \sigma^A. u \in g. \forall x = u \wedge \text{tgt} = v_{\text{tgt}} \wedge \sigma^A.\text{vis} = \emptyset \wedge \text{storegraph}(g, \sigma^A.\text{vis}, f_{\text{value}}) \} \\
 & \quad \text{dfs_find}(x, \text{tgt}) \\
 & \{ r^I. \exists b, \sigma^A. (r^I = \text{nullptr} \wedge b = \text{true} \wedge (\forall v. u \overset{g}{\rightsquigarrow} v \Leftrightarrow v \in \sigma^A.\text{vis}) \quad (\text{target not found}) \\
 & \quad \vee r^I \neq \text{nullptr} \wedge b = \text{false} \wedge f_{\text{value}}(r^I) = v_{\text{tgt}} \wedge u \overset{g}{\rightsquigarrow} r^I \quad (\text{target found}) \\
 & \quad \wedge \text{storegraph}(g, \sigma^A.\text{vis}, f_{\text{value}}) \}
 \end{aligned}$$

From this composed triple, we derive (11) as follows. For the precondition, $\exists \sigma^A. \sigma^A.\text{vis} = \emptyset \wedge \text{storegraph}(g, \sigma^A.\text{vis}, f_{\text{value}})$ simplifies to $\text{storegraph}(g, \emptyset, f_{\text{value}})$, matching the precondition of (11). For the postcondition, we take $p_{\text{vis}} = \sigma^A.\text{vis}$ as the witness for the existential in (11), and observe that b is entirely determined by r^I (via the disjunction), so the existential over b can be eliminated. This gives exactly (11).⁴

5 Discussion

When Does the Approach Help? The approach is most useful when the implementation relies on properties that are natural consequences of the algorithm but invisible to the implementation proof. In both of our case studies, the transferred properties (array bounds in KMP, vertex reachability in

⁴One could instead have tracked a separate concrete boolean mapping $f_{\text{vis}} : V \rightarrow \text{bool}$ related to vis via a correspondence predicate $\text{matched}(\text{vis}, f_{\text{vis}})$, but this would require maintaining the correspondence as an additional relational invariant and constructing an explicit witness for p_{vis} in the vertical composition step.

DFS) are exactly of this kind: they follow from the algorithm's invariants and would otherwise need to be re-established from scratch in the implementation proof. More generally, the approach applies whenever the two-layer method is used and the implementation's safety or functional correctness depends on intermediate facts about the abstract computation.

The cost of the approach is that the verifier must identify where to insert assertions. In our experience, the insertion points are not difficult to find: they correspond to the places where the implementation performs operations whose safety depends on the algorithm's behavior, such as array accesses or early termination decisions. The algorithm correctness proof already establishes the relevant invariants at these points; the assertion merely makes them explicit.

Beyond identifying the insertion points, the approach also assumes that the abstract and concrete programs admit an *alignment* at those points: a state-matching relation must let each inserted invariant be related to the concrete state where it is needed. This alignment does not require identical syntax or lockstep control flow; refinement frameworks can relate recursive and iterative formulations of the same algorithm [19]. When no such alignment exists between the abstract algorithm and the implementation, the assertion transfer may not apply, and either the abstract algorithm must be reorganized to admit one or the invariant must be established directly at the implementation level. In short, the trade-off is about the availability of alignment points: where they exist, the invariants established by the algorithm proof carry into the implementation proof unchanged; where they do not, those invariants must be re-established at the implementation level, or the abstract algorithm reshaped to admit alignment.

An Alternative Approach to the KMP Verification. In the KMP case study, the abstract algorithm uses the total list access $\text{nth } i \ l$ and is itself a total function. A natural question arises: could one instead use a partial list access to handle out-of-bounds cases directly in the abstract algorithm, dispensing with assertion annotations entirely? We compare this alternative approach with ours and show where it falls short.

The alternative replaces the total list access $\text{nth } i \ l$ with a partial list access $\text{nth_error}(i, l)$ of type $\mathbb{Z} \times \text{list } \mathbb{Z} \rightarrow \mathbb{Z}_\perp$, where $\perp$ represents undefined when i is out of bounds. This transforms the abstract algorithm into a partial function $\text{GetNext}_\perp^A$ of type $\text{list } \mathbb{Z} \rightarrow (\text{list } \mathbb{Z})_\perp$ that returns $\perp$ when any out-of-bounds access would occur during execution. Each list access must be guarded by an explicit $\perp$ check, as shown in Fig. 7.

One then defines a safety predicate $\text{Safe}(l) \triangleq \text{GetNext}_\perp^A(l) \neq \perp$ to capture when the algorithm succeeds. To prove implementation correctness, one must establish $\text{Safe}(l)$ as a precondition, yielding the following triple:

$$\begin{aligned} & \{ \text{Safe}(l) \wedge m = \text{len } l \wedge \text{storearray}(\text{str}, l) \} \\ & \quad \text{GetNext}(\text{str}, m) \\ & \{ r. \exists (l_r : \text{list } \mathbb{Z}). \text{GetNext}_\perp^A(l) = l_r \wedge \text{storearray}(\text{str}, l) * \text{storearray}(r, l_r) \} \end{aligned}$$

However, this alternative differs from ours in two ways. First, both approaches annotate the abstract algorithm with safety information, but in different places: our $\text{assert}(R^A)$ sits in the algorithm's body and is a no-op when R^A holds; the $\perp$ -check lifts the information into the return type, which becomes $(\text{list } \mathbb{Z})_\perp$ instead of $\text{list } \mathbb{Z}$, and inserts a conditional branch at every list access. Second, and more critically, $\text{Safe}(l)$ is an algorithm-level property, namely that list accesses remain within bounds, and the alternative requires it as a precondition of the implementation correctness proof. This entangles the two layers, breaking the separation of concerns that the two-layer method is meant to provide. Under our approach, by contrast, the implementation correctness proof receives the property via the ASSERT rule, while the algorithm correctness proof establishes it, keeping the two proofs independent.

Definition $\text{GetNext}^\perp l_{\text{str}} :=$

```

 $l_{\text{next}} := \{ 0 \}; i := 1; j := 0;$ 
while  $(i < \text{len } l_{\text{str}})$  {
    while  $(j > 0)$  {
        if  $\text{nth\_error } j \ l_{\text{str}} = \perp$  then return  $\perp$ ;
        if  $\text{nth\_error } i \ l_{\text{str}} = \perp$  then return  $\perp$ ;
        if  $\text{nth\_error } j \ l_{\text{str}} = \text{nth\_error } i \ l_{\text{str}}$  then break;
        if  $\text{nth\_error } (j - 1) \ l_{\text{next}} = \perp$  then return  $\perp$ ;
         $j := \text{nth\_error } (j - 1) \ l_{\text{next}}$ 
    }
    if  $\text{nth\_error } j \ l_{\text{str}} = \perp$  then return  $\perp$ ;
    if  $\text{nth\_error } i \ l_{\text{str}} = \perp$  then return  $\perp$ ;
    if  $\text{nth\_error } j \ l_{\text{str}} = \text{nth\_error } i \ l_{\text{str}}$ 
        then  $j := j + 1$ ;
     $l_{\text{next}} := l_{\text{next}} \uparrow\uparrow \{ j \}; i := i + 1$ ;
return  $l_{\text{next}}$ 
    
```

Fig. 7. The modified abstract algorithm $\text{GetNext}^\perp$ using partial list access nth_error . Highlighted lines are added $\perp$ checks that do not appear in the original algorithm; they clutter the algorithm with guards for conditions that the KMP invariant guarantees never occur.

6 Related Work

Verification of complex programs has been approached from many angles. We discuss two lines of work most relevant to ours: functional correctness of imperative programs, and relational Hoare logic frameworks.

Functional Correctness for Complex Programs. Wang et al. [27] developed a graph library in Rocq. Subsequently, Mohan et al. [23] extended this library to support undirected graphs. Their line of work directly proves the functional correctness of C programs that manipulate graph data structures using standard Hoare logic. In contrast, this pearl proves functional correctness through the two-layer method, separating implementation correctness from algorithm correctness. This allows the algorithm correctness proof to be reused across different concrete implementations: we prove that both a recursive DFS vertex finding implementation and a DFS sum implementation refine the same abstract DFS description. Our work builds on their graph definitions, enabling the algorithm correctness proof to leverage existing theorems in graph theory. Cao et al. [11] proved the functional correctness of an imperative insertion function for binary search trees with the two-layer method [2]. Meanwhile, LAProof [16] offers a specialized library for verifying linear algebra programs, featuring case studies such as matrix multiplication. However, they both employ standard (separation) Hoare logic to establish implementation correctness. In this pearl, we employ relational Hoare logic instead, since implementation correctness is essentially a relation between a concrete program and its abstract counterpart. As Beckert and Ulbrich observe [6], relational verification is often simpler than functional verification when the two programs are similar, which is precisely the case when a concrete implementation closely follows an abstract algorithm.

Building on the Isabelle Refinement Framework [20], Lammich and Neumann [19] proposed a layered method to prove the functional correctness of DFS-based abstract functional algorithms. With Imperative HOL [10], this framework enables one to first verify the correctness of algorithm descriptions, which are purely functional, and then refine that abstract description step by step at the monad layer. The final result is an executable imperative program. While we both rely on

refinement to prove implementation correctness, we employ relational quadruples where abstract programs are modeled as state relation monads with error-exit mechanisms.

Relational Hoare Logic Frameworks. Relational Hoare logic, introduced by Benton [7], extends traditional Hoare logic to relate two programs. Many relational verification frameworks have been proposed to verify relational properties, including equivalence [31, 18, 28, 21, 4] and refinement [5, 13]. This pearl focuses on verifying program refinement, which can be formally captured via simulation relations. Aguirre et al. [1] proposed a unary higher-order logic (UHOL) and a relational higher-order logic (RHOL) for reasoning about relational properties of a simply typed λ -calculus. RHOL provides inference rules [UHOL-L] and [UHOL-R] to allow RHOL proofs to fall back to UHOL reasoning. These two rules serve a role similar to the focusing rules (**FOCUS-LEFT** and **FOCUS-RIGHT**) in this pearl, enabling independent evaluation on one side. The work of Gladshstein et al. [14] focuses on verifying the correctness of operations where identical or sparse data are stored in arrays through compression or other space-efficient representations. Such implementations often involve multiple loops with intricate invariants. The authors argue that relational program logics are well-suited to address these verification tasks, an assessment that aligns with our own perspective. Relational Hoare logic can offer an intuitive and convenient framework for proving implementation correctness. However, they introduce the framework LGTM (Logic for Graceful Tensor Manipulation), which dynamically determines the number k of traces to compare, allowing k to depend on program variables, while we rely on relational Hoare quadruples. Moreover, none of these frameworks focus on fully separating algorithm correctness from implementation correctness and employing assertion annotations in programs, which are central to our approach.

Song et al. [25] proposed the use of `assume` and `assert` statements to encode the preconditions and postconditions of functions, thereby proving refinement under the condition that some functions satisfy these preconditions and postconditions. While their method uses assertion annotations to encode function contracts, our approach differs in several key aspects. Their assertions are primarily used to express preconditions and postconditions of functions, whereas our assertions can appear at any point in the abstract program to capture key invariants. For instance, in our KMP example, we insert assertions before entering the while loop to ensure memory safety for implementation proof. Furthermore, while they focus on proving refinement by checking that specific functions satisfy the encoded conditions, we aim to separate algorithm correctness from implementation correctness and avoid redundant proofs. Additionally, we aim to derive functional correctness for the concrete implementation, which is achieved using vertical composition rules. Several works have proposed automated approaches for relational verification. Unno et al. [26] present a constraint-based approach to relational verification that supports a variety of relational properties, including refinement. Beutner and Finkbeiner [9] propose an automated method for verifying $\forall^*\exists^*$ hyperproperties, and Itzhaky et al. [15] reduce hyperproperty verification to the satisfiability of constrained Horn clauses (CHCs). In this pearl, we concentrate on interactive verification of refinement properties. Nonetheless, techniques developed in those works may complement our method in future integration efforts.

Assertions in Monads. Modeling assertions as monadic operations is not new. Benton, Hughes, and Moggi [8] study monads and effects in a general setting, and Chitil and Huch [12] develop monadic assertions in Haskell. Our use of `assert`(P^A) as a monadic operation is in the same spirit. The novelty of this pearl lies not in the mechanism itself, but in how assertion annotations are used within the two-layer verification method: by inserting assertions into the abstract program at invariant points, we enable properties established in the algorithm proof to transfer directly to the implementation proof via the error simulation condition of relational Hoare quadruples.

7 Conclusion and Future Work

In this pearl, we enhance the two-layer method by achieving a clean separation between algorithm correctness and implementation correctness. We model abstract algorithms as state relation monads with error-exit mechanisms and use relational Hoare logic to relate them to concrete implementations. Assertion annotations inserted into the abstract program transfer invariants directly to the implementation proof, eliminating redundant proof effort. We demonstrate the approach through two case studies mechanized in Rocq: the KMP next-array computation and a DFS vertex search. The lesson is that in two-layer verification, the abstract program is not just a specification to be refined, it could be a channel through which invariants flow.

Several directions remain for future work. First, this pearl focuses on partial correctness and does not address nontermination. One possible approach to total correctness is to enrich the abstract program state with time credits and prove that the program terminates within a given budget; if the budget reaches zero, the program produces an error. While this can be encoded in our framework, a more principled treatment of nontermination remains an important direction. Second, assertion annotations are currently inserted manually by the verifier. Automating this process, for instance by inferring annotation points from the algorithm correctness proof, would reduce the proof burden and make the approach more accessible. Third, the abstract algorithm itself is currently user-written. Automating its generation is a direction we would like to pursue.

Acknowledgments

We thank the anonymous ICFP reviewers for their careful reading and constructive feedback, which substantially improved the presentation of this pearl. This work was supported by the National Natural Science Foundation of China under Grant No. 62472274.

A State Relation Monads and Implementations

The state relation monad with error exit mechanisms $\text{St}^E R$ consists of two components. For a monad $c^A \in \text{St}^E R$, $\llbracket c^A \rrbracket_{\text{nrm}} \subseteq \Sigma^A \times R \times \Sigma^A$ collects all possible state transitions, and $\llbracket c^A \rrbracket_{\text{err}} \subseteq \Sigma^A$ collects all states from which the program terminates with an error:

- $(\sigma_1^A, r, \sigma_2^A) \in \llbracket c^A \rrbracket_{\text{nrm}}$: from state σ_1^A the program c^A may terminate at state σ_2^A and return value r ;
- $\sigma_1^A \in \llbracket c^A \rrbracket_{\text{err}}$: from state σ_1^A the program c^A results in an error.

Then the basic monadic operation $\text{return } r$ returns the value r without changing the state, and will never exit with errors.

$$\begin{array}{lcl} (\sigma_1^A, r, \sigma_2^A) & \in \llbracket \text{return } r_0 \rrbracket_{\text{nrm}} & \text{iff. } \sigma_1^A = \sigma_2^A \text{ and } r = r_0 \\ \sigma_1^A & \notin \llbracket \text{return } r_0 \rrbracket_{\text{err}} & \end{array}$$

The basic monadic operation $\text{bind } c^A f^A$ represents the chaining of two monadic computations. The first computation c^A may trigger an error or produce a result r_0 and update the state from σ_1^A to σ_0^A . The second computation f^A is then applied to the result r_0 and potentially triggers an error or produces a new state and result.

$$\begin{array}{lcl} (\sigma_1^A, r, \sigma_2^A) & \in \llbracket \text{bind } c^A f^A \rrbracket_{\text{nrm}} & \text{iff. } \exists \sigma_0^A r_0 \text{ s.t. } (\sigma_1^A, r_0, \sigma_0^A) \in \llbracket c^A \rrbracket_{\text{nrm}} \text{ and} \\ & & (\sigma_0^A, r, \sigma_2^A) \in \llbracket f^A(r_0) \rrbracket_{\text{nrm}} \\ \sigma_1^A & \in \llbracket \text{bind } c^A f^A \rrbracket_{\text{err}} & \text{iff. } \sigma_1^A \in \llbracket c^A \rrbracket_{\text{err}} \text{ or } \exists \sigma_0^A r_0 \text{ s.t.} \\ & & (\sigma_1^A, r_0, \sigma_0^A) \in \llbracket c^A \rrbracket_{\text{nrm}} \text{ and } \sigma_0^A \in \llbracket f^A(r_0) \rrbracket_{\text{err}} \end{array}$$

For convenience, we adopt the following notations:

$$x \leftarrow c^A; f^A(x) \triangleq \text{bind } c^A (\lambda x. f^A(x))$$

$$c_1^A; c_2^A \triangleq \text{bind } c_1^A (\lambda _. c_2^A)$$

PROPOSITION A.1. *The two basic monad constructs obey the following three rules:*

- $\text{bind } (\text{return } r) f^A \equiv f^A r$
- $\text{bind } c^A (\lambda x. \text{return } x) \equiv c^A$
- $\text{bind } c^A (\lambda x. \text{bind } (f^A x) g^A) \equiv \text{bind } (\text{bind } c^A f^A) g^A$

The three rules in the proposition describe key properties of monadic operations. The first rule shows that binding a return statement to a function is equivalent to directly applying the function to the value, while the second rule indicates that binding a computation to a function returning a constant value has no effect on the computation. The third rule expresses the associativity of the bind operation, ensuring that the order of nested binds does not affect the result.

Besides, we include the condition operation $\text{assume}(P^A)$ and assertion operation $\text{assert}(P^A)$, both of which succeed and return the unit value $()$ if and only if the predicate P^A holds for the current state. However, they differ in their behavior when the predicate does not hold: $\text{assume}(P^A)$ blocks and $\text{assert}(P^A)$ trigger an error.

$$\begin{array}{ll} (\sigma_1^A, r, \sigma_2^A) \in \llbracket \text{assume}(P^A) \rrbracket_{\text{norm}} & \text{iff. } \sigma_1^A = \sigma_2^A, \sigma_1^A \in P^A \text{ and } r = () \\ \sigma_1^A \notin \llbracket \text{assume}(P^A) \rrbracket_{\text{err}} & \\ (\sigma_1^A, r, \sigma_2^A) \in \llbracket \text{assert}(P^A) \rrbracket_{\text{norm}} & \text{iff. } \sigma_1^A = \sigma_2^A, \sigma_1^A \in P^A \text{ and } r = () \\ \sigma_1^A \in \llbracket \text{assert}(P^A) \rrbracket_{\text{err}} & \text{iff. } \sigma_1^A \notin P^A \end{array}$$

To model nondeterministic branches, we use $\text{NDchoice}(c_1^A, c_2^A)$:

$$\begin{array}{ll} (\sigma_1^A, r, \sigma_2^A) \in \llbracket \text{NDchoice}(c_1^A, c_2^A) \rrbracket_{\text{norm}} & \text{iff. } (\sigma_1^A, r, \sigma_2^A) \in \llbracket c_1^A \rrbracket_{\text{norm}} \text{ or } (\sigma_1^A, r, \sigma_2^A) \in \llbracket c_2^A \rrbracket_{\text{norm}} \\ \sigma_1^A \in \llbracket \text{NDchoice}(c_1^A, c_2^A) \rrbracket_{\text{err}} & \text{iff. } \sigma_1^A \in \llbracket c_1^A \rrbracket_{\text{err}} \text{ or } \sigma_1^A \in \llbracket c_2^A \rrbracket_{\text{err}} \end{array}$$

These primitives enable us to define traditional branching constructs. In particular, we specify if-then and if-else statements as follows:

$$\begin{array}{l} \text{if } P^A \text{ then } c^A \triangleq \text{NDchoice}(\text{assume}(P^A); c^A, \text{assume}(\neg P^A); \text{return } ()) \\ \text{if } P^A \text{ then } c_1^A \text{ else } c_2^A \triangleq \text{NDchoice}(\text{assume}(P^A); c_1^A, \text{assume}(\neg P^A); c_2^A) \end{array}$$

A.1 Support for General Recursions and Loops

To support recursive functions and loops for state relation monads, and state relation monads with error handling, we rely on a fixed-point operator fix . The semantic interpretation of a program is modeled as a subset of an appropriate domain: $\Sigma^A \times R \times \Sigma^A$ (for normal termination behaviors) and Σ^A (for error behaviors). Then we define the order of state relation monads;

$$c_1^A \leq c_2^A \quad \text{iff.} \quad \llbracket c_1^A \rrbracket_{\text{norm}} \subseteq \llbracket c_2^A \rrbracket_{\text{norm}} \text{ and } \llbracket c_1^A \rrbracket_{\text{err}} \subseteq \llbracket c_2^A \rrbracket_{\text{err}}$$

Then state relation monads form complete partially ordered sets under this order, which allows us to apply Kleene's fixed-point theorem [29]. Any continuous and monotonic function F on such a domain admits a least fixed point, given by the supremum of the chain obtained by iteratively applying F to the bottom element $\perp$:

$$\text{fix}(F) \triangleq \bigcup_{n=0}^{\infty} F^n(\perp)$$

where $\perp$ is interpreted as $\llbracket \perp \rrbracket_{\text{norm}} = \emptyset$ and $\llbracket \perp \rrbracket_{\text{err}} = \emptyset$. We thus define general recursive functions and loops by first specifying such a function F , and then applying fix to obtain the least fixed point $f \triangleq \text{fix}(F)$. This function f is guaranteed to satisfy the fixed-point equation $f = F(f)$ if f is continuous and monotonic. Then we define a while loop statement $\text{while } (b^A) \{c^A\}$ and a repetition statement $\text{Repeat}\{c^A\}$. We formalize each by first defining a fixpoint function, and then employing the fixed-point operator fix .

Context $\{R \text{ A: Type}\}$.

Definition $F_{\text{while}} (P: R \rightarrow \mathcal{P}(\Sigma^A)) (c: R \rightarrow \text{St}^E R)$

$(X: R \rightarrow \text{St}^E R) : R \rightarrow \text{St}^E R :=$

fun $r \Rightarrow$

NDchoice (**assume**($P \ r$) ; $r_0 \leftarrow c \ r$; $X \ r_0$,

assume($\neg (P \ r)$) ; **return** r).

Definition $\text{while } (P: R \rightarrow \mathcal{P}(\Sigma^A)) (c: R \rightarrow \text{St}^E R) := \text{fix } (F_{\text{while}} \ P \ c)$.

(* F_{while} is monotonic and continuous *)

Theorem while_unfold : **forall** $b (c: \text{St}^E R)$,

while $b \ c = F_{\text{while}} \ b \ c$ (**while** $b \ c$).

Definition $F_{\text{repeatbreak}} (c: A \rightarrow \text{St}^E (A + R)) (X: A \rightarrow \text{St}^E R) : A \rightarrow \text{St}^E R :=$

fun $a \Rightarrow$

$x \leftarrow c \ a$;

(**match** x **with**

| $\text{inl } a_0 \Rightarrow X \ a_0$ | $\text{inr } r \Rightarrow \text{return } r$ **end**).

Definition $\text{Repeat } (c: A \rightarrow \text{St}^E (A + R)) := \text{fix } (F_{\text{repeatbreak}} \ c)$.

(* $F_{\text{repeatbreak}}$ is monotonic and continuous *)

Theorem repeat_unfold : **forall** $(c: A \rightarrow \text{St}^E (A + R))$,

Repeat $c = F_{\text{repeatbreak}} \ c$ (**Repeat** c).

Then, using these constructs, we can formalize the KMP and DFS algorithms. For example, the inner loop for computing the “next” array algorithm can be formalized as follows:

Context $(l_{\text{str}}: \text{list } Z)$.

Context $(l_{\text{next}}: \text{list } Z)$.

Context $(i: Z)$.

Definition $\text{inner_body} : Z \rightarrow \text{St}^E Z :=$

fun $j \Rightarrow \text{assert } (j < \text{len } (l_{\text{next}})) ;$

let $j_0 := \text{nth } (j - 1) \ l_{\text{next}}$ **in**

$\text{assert } (j_0 < \text{len } (l_{\text{str}}))$; **return** j_0 .

Definition $\text{inner_loop } (j: Z) : \text{St}^E Z :=$

while (**fun** $j' \Rightarrow j' > 0 \wedge \text{nth } j \ l_{\text{next}} \neq \text{nth } i \ l_{\text{next}}$) $\text{inner_body } j$.

B Relational Hoare Rules

Here, we present a set of relational Hoare rules for deriving a quadruple that relates a C program and a state relation monad.

$$\begin{array}{c}
\text{RET} \frac{}{\{\Phi \ u v\} \text{ return } u \preceq \text{ return } v \ \{\Phi\}} \qquad \text{EQUIV} \frac{c_1^A \equiv c_2^A \quad \{\mathcal{P}\} \ c^I \preceq c_2^A \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq c_1^A \ \{\Phi\}} \\
\\
\text{ASGN} \frac{}{\{(\Phi \ v)[e/x]\} \ x := e \preceq \text{ return } v \ \{r^A . \Phi \ r^A\}} \\
\\
\text{STORE} \frac{}{\{\Phi \ v * l \mapsto _ \} \ \text{store } l \ v \preceq \text{ return } v \ \{r^A . (\Phi \ r^A) * l \mapsto v\}} \\
\\
\text{LOAD} \frac{}{\{(\Phi \ v)[u/x] * l \mapsto u\} \ x := !l \preceq \text{ return } v \ \{r^A . (\Phi \ r^A) * l \mapsto u\}} \\
\\
\text{FOCUS-LEFT} \frac{\{\mathcal{P}\} \ c_1^I \preceq \text{ return } () \ \{\mathcal{R}\} \quad \{\mathcal{R}\} \ c_2^I \preceq c^A \ \{\Phi\}}{\{\mathcal{P}\} \ c_1^I; c_2^I \preceq c^A \ \{\Phi\}} \\
\\
\text{FOCUS-RIGHT} \frac{\{\mathcal{P}\} \ \text{skip} \preceq c^A \ \{r^A . \Psi \ r^A\} \quad \forall v. \{\Psi \ v\} \ c^I \preceq f^A v \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq \text{bind } c^A \ f^A \ \{\Phi\}} \\
\\
\text{SEQ-SIM} \frac{\{\mathcal{P}\} \ c_1^I \preceq c^A \ \{r^A . \Psi \ r^A\} \quad \forall v. \{\Psi \ v\} \ c_2^I \preceq f^A v \ \{\Phi\}}{\{\mathcal{P}\} \ c_1^I; c_2^I \preceq \text{bind } c^A \ f^A \ \{\Phi\}} \\
\\
\text{WHILE-SIM} \frac{\forall u. \{\Psi \ u \wedge b^I \wedge b^A u\} \ c^I \preceq c^A v \ \{r^A . \Psi \ r^A\} \quad \forall w. \Psi \ w \Rightarrow (b^I \Leftrightarrow b^A w)}{\{\Psi \ v\} \ \text{while } b^I \ \text{do } c^I \preceq \text{while } b^A \ \text{do } c^A v \ \{r^A . \Psi \ r^A \wedge \neg b^I \wedge \neg b^A\}} \\
\\
\text{REPEAT-SIM} \frac{\forall u. \{\Psi \ u \wedge b^I\} \ c^I \preceq c^A u \ \{r^A . (\exists v. r^A = \text{inl}(v) \wedge \Psi \ v) \vee (\exists v. r^A = \text{inr}(v) \wedge \Phi \ v)\}}{\{\Psi \ v\} \ \text{while } b^I \ \text{do } c^I \preceq \text{repeat } c^A v \ \{r^A . \Phi \ r^A \wedge \neg b^I\}} \\
\\
\text{ASSUME} \frac{\mathcal{P} \Rightarrow R^A \quad \{\mathcal{P}\} \ c^I \preceq c^A \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq \text{bind assume}(R^A) \ c^A \ \{\Phi\}} \qquad \text{ASSERT} \frac{\{\mathcal{P} \wedge R^A\} \ c^I \preceq c^A \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq \text{bind assert}(R^A) \ c^A \ \{\Phi\}} \\
\\
\text{ND-LEFT} \frac{\{\mathcal{P}\} \ c^I \preceq c_1^A \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq \text{NDchoice}(c_1^A, c_2^A) \ \{\Phi\}} \qquad \text{ND-RIGHT} \frac{\{\mathcal{P}\} \ c^I \preceq c_2^A \ \{\Phi\}}{\{\mathcal{P}\} \ c^I \preceq \text{NDchoice}(c_1^A, c_2^A) \ \{\Phi\}} \\
\\
\text{IFE} \frac{\begin{array}{c} \{\mathcal{P} \wedge b^I\} \ c_1^I \preceq c_1^A \ \{\Phi\} \quad \{\mathcal{P} \wedge (\neg b^I)\} \ c_2^I \preceq c_2^A \ \{\Phi\} \quad \text{If } \mathcal{P} \Rightarrow (b^I \Leftrightarrow b^A) \quad \{\mathcal{P} \wedge b^I\} \ c_1^I \preceq c_2^A \ \{\Phi\} \\ \{\mathcal{P} \wedge (\neg b^I)\} \ c_2^I \preceq c_1^A \ \{\Phi\} \quad \text{If } \mathcal{P} \Rightarrow (b^I \Leftrightarrow \neg b^A) \quad \{\mathcal{P} \wedge b^I\} \ c_1^I \preceq c_1^A \ \{\Phi\} \quad \{\mathcal{P} \wedge (\neg b^I)\} \ c_2^I \preceq c_1^A \ \{\Phi\} \\ \text{If } \mathcal{P} \Rightarrow (b^I \vee \neg b^I \Rightarrow b^A) \quad \{\mathcal{P} \wedge b^I\} \ c_1^I \preceq c_2^A \ \{\Phi\} \quad \{\mathcal{P} \wedge (\neg b^I)\} \ c_2^I \preceq c_2^A \ \{\Phi\} \quad \text{If } \mathcal{P} \Rightarrow (b^I \vee \neg b^I \Rightarrow \neg b^A) \end{array}}{\{\mathcal{P}\} \ \text{if } b^I \ \text{then } c_1^I \ \text{else } c_2^I \preceq \text{if } b^A \ \text{then } c_1^A \ \text{else } c_2^A \ \{\Phi\}} \\
\\
\text{IFE-LEFT} \frac{\{\mathcal{P} \wedge b^I\} \ c_1^I \preceq c^A \ \{\Phi\} \quad \{\mathcal{P} \wedge \neg b^I\} \ c_2^I \preceq c^A \ \{\Phi\}}{\{\mathcal{P}\} \ \text{if } b^I \ \text{then } c_1^I \ \text{else } c_2^I \preceq c^A \ \{\Phi\}}
\end{array}$$

C Addressing Recursive Calls

To reason about function calls, we extend the implementation language by parameterizing it with an environment χ , which describes the behavior of function invocations. Let Fn denote the set of function names. Then, an environment χ maps each function to its semantics.

While standard contextual Hoare logic facilitates modular reasoning about function calls on the imperative side through proof contexts, we then extend the relational Hoare Quadruples with proof contexts to handle function calls. Relational specifications capture how the behavior of a concrete program function relates to some state relation monad.

Definition C.1 (Relational Function Specification). *A relational specification is parameterized by variables of any type and has the form*

$$(f^I : R^I.R^A.\Pi(\vec{a} : A). \{f^A \vec{a}\} \{\mathcal{P} \vec{a}\} \{Q \vec{a}\}).$$

Here, $\vec{a}$ is a list of the logical variables, the monad program $f^A \vec{a}$ has type $\text{St}^E R^A$, the precondition $\mathcal{P} \vec{a} \subseteq \Sigma^I \times \Sigma^A$, and the postcondition $Q \vec{a} \subseteq R^I \times R^A \times \Sigma^I \times \Sigma^A$.

We use $S_{V\exists}$ with a subscript $\forall\exists$ to denote the proof context that consists of relational function specifications.

Definition C.2 (Relational Valid Environment). *Environment χ is relationally valid with respect to $S_{V\exists}$, denoted as $\text{RelValid}(\chi, S_{V\exists})$, if for every specification*

$$(f_i^I : R_i^I.R_i^A.\Pi(\vec{a} : A_i). \{f_i^A \vec{a}\} \{\mathcal{P}_i \vec{a}\} \{Q_i \vec{a}\})$$

in $S_{V\exists}$ and given any logical variable $\vec{a} \in A_i$, if $(\sigma_1^I, \sigma_1^A) \models \mathcal{P}_i \vec{a}$, then according to environment χ

- if f_i^I results in an error, then $\sigma_1^A \in \llbracket f_i^A \vec{a} \rrbracket_{\text{err}}$
- if there exists an execution of implementation f^A with state σ^I and return value r^I , then either $\sigma_1^A \in \llbracket f_i^A \vec{a} \rrbracket_{\text{err}}$ or there exist σ_2^A and r^A such that $(\sigma_1^A, r^A, \sigma_2^A) \in \llbracket f_i^A \vec{a} \rrbracket_{\text{nm}}$ and $(\sigma_2^I, \sigma_2^A) \models Q_i \vec{a} \ r^I \ r^A$.

Then we have the following relational call rule:

$$\text{REL-CALL} \frac{(f^I : R^I.R^A.\Pi(\vec{a} : A). \{f^A \vec{a}\} \{\mathcal{P} \vec{a}\} \{Q \vec{a}\}) \in S_{V\exists}}{S_{V\exists} \vdash \{\mathcal{P} \vec{a}\} \text{ call } f^I \lesssim f^A \vec{a} \{r^I, r^A. Q \vec{a} \ r^I \ r^A\}}$$

References

- [1] Alejandro Aguirre, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Pierre-Yves Strub. 2017. A relational logic for higher-order programs. *Proc. ACM Program. Lang.*, 1, ICFP, 21:1–21:29. doi:[10.1145/3110265](https://doi.org/10.1145/3110265) (cit. on p. 16).
- [2] Andrew W. Appel. 2022. Coq’s vibrant ecosystem for verification engineering (invited talk). In *CPP ’22: 11th ACM SIGPLAN International Conference on Certified Programs and Proofs, Philadelphia, PA, USA, January 17 - 18, 2022*. Andrei Popescu and Steve Zdancewic, (Eds.) ACM, 2–11. doi:[10.1145/3497775.3503951](https://doi.org/10.1145/3497775.3503951) (cit. on pp. 1, 15).
- [3] Andrew W. Appel. 2026. *Verified Functional Algorithms*. Benjamin C. Pierce, (Ed.) *Software Foundations*. Vol. 3. Version 1.6.0. Electronic textbook. <https://softwarefoundations.cis.upenn.edu/vfa-current/index.html> (cit. on p. 2).
- [4] Anindya Banerjee, Ramana Nagasamudram, David A. Naumann, and Mohammad Nikouei. 2022. A relational program logic with data abstraction and dynamic framing. *ACM Trans. Program. Lang. Syst.*, 44, 4, 25:1–25:136. doi:[10.1145/3551497](https://doi.org/10.1145/3551497) (cit. on p. 16).
- [5] Gilles Barthe, Juan Manuel Crespo, and César Kunz. 2013. Beyond 2-safety: asymmetric product programs for relational program verification. In *Logical Foundations of Computer Science, International Symposium, LFCS 2013, San Diego, CA, USA, January 6-8, 2013. Proceedings* (Lecture Notes in Computer Science). Sergei N. Artëmov and Anil Nerode, (Eds.) Vol. 7734. Springer, 29–43. doi:[10.1007/978-3-642-35722-0_3](https://doi.org/10.1007/978-3-642-35722-0_3) (cit. on p. 16).
- [6] Bernhard Beckert and Matthias Ulbrich. 2018. Trends in relational program verification. In *Principled Software Development - Essays Dedicated to Arnd Poetzsch-Heffter on the Occasion of his 60th Birthday*. Peter Müller and Ina Schaefer, (Eds.) Springer, 41–58. doi:[10.1007/978-3-319-98047-8_3](https://doi.org/10.1007/978-3-319-98047-8_3) (cit. on p. 15).
- [7] Nick Benton. 2004. Simple relational correctness proofs for static analyses and program transformations. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2004, Venice, Italy, January 14-16, 2004*. Neil D. Jones and Xavier Leroy, (Eds.) ACM, 14–25. doi:[10.1145/964001.964003](https://doi.org/10.1145/964001.964003) (cit. on pp. 3, 5, 16).
- [8] Nick Benton, John Hughes, and Eugenio Moggi. 2000. Monads and effects. In *Applied Semantics, International Summer School, APPSEM 2000, Caminha, Portugal, September 9-15, 2000, Advanced Lectures* (Lecture Notes in Computer Science). Gilles Barthe, Peter Dybjer, Luís Pinto, and João Saraiva, (Eds.) Vol. 2395. Springer, 42–122. doi:[10.1007/3-540-45699-6_2](https://doi.org/10.1007/3-540-45699-6_2) (cit. on p. 16).
- [9] Raven Beutner and Bernd Finkbeiner. 2022. Software verification of hyperproperties beyond k-safety. In *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part I* (Lecture Notes in Computer Science). Sharon Shoham and Yakir Vizel, (Eds.) Vol. 13371. Springer, 341–362. doi:[10.1007/978-3-031-13185-1_17](https://doi.org/10.1007/978-3-031-13185-1_17) (cit. on p. 16).
- [10] Lukas Bulwahn, Alexander Krauss, Florian Haftmann, Levent Erkök, and John Matthews. 2008. Imperative functional programming with isabelle/hol. In *Theorem Proving in Higher Order Logics*. Otmane Ait Mohamed, César Muñoz, and Sofiène Tahar, (Eds.) Springer Berlin Heidelberg, Berlin, Heidelberg, 134–149. ISBN: 978-3-540-71067-7. doi:[10.1007/978-3-540-71067-7_14](https://doi.org/10.1007/978-3-540-71067-7_14) (cit. on p. 15).
- [11] Qinxiang Cao, Shengyi Wang, Aquinas Hobor, and Andrew W Appel. 2019. Proof pearl: magic wand as frame. *arXiv preprint arXiv:1909.08789*. doi:[10.48550/arXiv.1909.08789](https://doi.org/10.48550/arXiv.1909.08789) (cit. on pp. 1, 15).
- [12] Olaf Chitil and Frank Huch. 2007. Monadic, prompt lazy assertions in haskell. In *Programming Languages and Systems - 5th Asian Symposium, APLAS 2007*. Springer, 38–53. doi:[10.1007/978-3-540-76637-7_4](https://doi.org/10.1007/978-3-540-76637-7_4) (cit. on p. 16).
- [13] Martin Clochard, Claude Marché, and Andrei Paskevich. 2020. Deductive verification with ghost monitors. *Proc. ACM Program. Lang.*, 4, POPL, 2:1–2:26. doi:[10.1145/3371070](https://doi.org/10.1145/3371070) (cit. on p. 16).
- [14] Vladimir Gladshstein, Qiyuan Zhao, Willow Ahrens, Saman P. Amarasinghe, and Ilya Sergey. 2024. Mechanised hypersafety proofs about structured data. *Proc. ACM Program. Lang.*, 8, PLDI, 647–670. doi:[10.1145/3656403](https://doi.org/10.1145/3656403) (cit. on p. 16).
- [15] Shachar Itzhaky, Sharon Shoham, and Yakir Vizel. 2024. Hyperproperty verification as CHC satisfiability. In *Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part II* (Lecture Notes in Computer Science). Stephanie Weirich, (Ed.) Vol. 14577. Springer, 212–241. doi:[10.1007/978-3-031-57267-8_9](https://doi.org/10.1007/978-3-031-57267-8_9) (cit. on p. 16).
- [16] Ariel E. Kellison, Andrew W. Appel, Mohit Tekriwal, and David Bindel. 2023. Laproof: a library of formal proofs of accuracy and correctness for linear algebra programs. In *2023 IEEE 30th Symposium on Computer Arithmetic (ARITH)*, 36–43. doi:[10.1109/ARITH58626.2023.00021](https://doi.org/10.1109/ARITH58626.2023.00021) (cit. on p. 15).
- [17] Bram Kohlen, Maximilian Schäffeler, Mohammad Abdulaziz, Arnd Hartmanns, and Peter Lammich. 2025. A formally verified IEEE 754 floating-point implementation of interval iteration for mdps. In *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part II* (Lecture Notes in Computer Science). Ruzica Piskac and Zvonimir Rakamaric, (Eds.) Vol. 15932. Springer, 122–146. doi:[10.1007/978-3-031-98679-6_6](https://doi.org/10.1007/978-3-031-98679-6_6) (cit. on p. 10).

- [18] Sudipta Kundu, Zachary Tatlock, and Sorin Lerner. 2009. Proving optimizations correct using parameterized program equivalence. In *Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2009, Dublin, Ireland, June 15-21, 2009*. Michael Hind and Amer Diwan, (Eds.) ACM, 327–337. doi:[10.1145/1542476.1542513](https://doi.org/10.1145/1542476.1542513) (cit. on p. 16).
- [19] Peter Lammich and René Neumann. 2015. A framework for verifying depth-first search algorithms. In *Proceedings of the 2015 Conference on Certified Programs and Proofs*, 137–146. doi:[10.1145/2676724.2693165](https://doi.org/10.1145/2676724.2693165) (cit. on pp. 14, 15).
- [20] Peter Lammich and Thomas Tuerk. 2012. Applying data refinement for monadic programs to hopcroft’s algorithm. In *International Conference on Interactive Theorem Proving*. Springer, 166–182. doi:[10.1007/978-3-642-32347-8_12](https://doi.org/10.1007/978-3-642-32347-8_12) (cit. on p. 15).
- [21] Kenji Maillard, Catalin Hritcu, Exequiel Rivas, and Antoine Van Muylder. 2020. The next 700 relational program logics. *Proc. ACM Program. Lang.*, 4, POPL, 4:1–4:33. doi:[10.1145/3371072](https://doi.org/10.1145/3371072) (cit. on p. 16).
- [22] Eugenio Moggi. 1991. Notions of computation and monads. *Inf. Comput.*, 93, 1, 55–92. doi:[10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4) (cit. on p. 4).
- [23] Anshuman Mohan, Wei Xiang Leow, and Aquinas Hobor. 2021. Functional correctness of C implementations of dijkstra’s, kruskal’s, and prim’s algorithms. In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II* (Lecture Notes in Computer Science). Alexandra Silva and K. Rustan M. Leino, (Eds.) Vol. 12760. Springer, 801–826. doi:[10.1007/978-3-030-81688-9_37](https://doi.org/10.1007/978-3-030-81688-9_37) (cit. on p. 15).
- [24] John C. Reynolds. 2002. Separation logic: A logic for shared mutable data structures. In *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. IEEE Computer Society, 55–74. doi:[10.1109/LICS.2002.1029817](https://doi.org/10.1109/LICS.2002.1029817) (cit. on p. 2).
- [25] Youngju Song, Minki Cho, Dongjae Lee, Chung-Kil Hur, Michael Sammler, and Derek Dreyer. 2023. Conditional contextual refinement. *Proc. ACM Program. Lang.*, 7, POPL, 1121–1151. doi:[10.1145/3571232](https://doi.org/10.1145/3571232) (cit. on p. 16).
- [26] Hiroshi Unno, Tachio Terauchi, and Eric Koskinen. 2021. Constraint-based relational verification. In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I* (Lecture Notes in Computer Science). Alexandra Silva and K. Rustan M. Leino, (Eds.) Vol. 12759. Springer, 742–766. doi:[10.1007/978-3-030-81685-8_35](https://doi.org/10.1007/978-3-030-81685-8_35) (cit. on p. 16).
- [27] Shengyi Wang, Qinxiang Cao, Anshuman Mohan, and Aquinas Hobor. 2019. Certifying graph-manipulating C programs via localizations within data structures. *Proc. ACM Program. Lang.*, 3, OOPSLA, 171:1–171:30. doi:[10.1145/3360597](https://doi.org/10.1145/3360597) (cit. on p. 15).
- [28] Yuepeng Wang, Isil Dillig, Shuvendu K. Lahiri, and William R. Cook. 2018. Verifying equivalence of database-driven applications. *Proc. ACM Program. Lang.*, 2, POPL, 56:1–56:29. doi:[10.1145/3158144](https://doi.org/10.1145/3158144) (cit. on p. 16).
- [29] Glynn Winskel. 1993. *The formal semantics of programming languages - an introduction*. Foundation of computing series. MIT Press. ISBN: 978-0-262-23169-5 (cit. on p. 18).
- [30] [SW] Shushu Wu, Chengxi Yang, Xiwei Wu, and Qinxiang Cao, Assertions for Free: Transferring Invariants from Algorithm to Implementation Proofs June 2026. doi:[10.5281/zenodo.20540814](https://doi.org/10.5281/zenodo.20540814), URL: <https://doi.org/10.5281/zenodo.20540814> (cit. on p. 4).
- [31] Hongseok Yang. 2007. Relational separation logic. *Theor. Comput. Sci.*, 375, 1-3, 308–334. doi:[10.1016/J.TCS.2006.12.036](https://doi.org/10.1016/J.TCS.2006.12.036) (cit. on pp. 5, 16).

Received 2026-02-19; accepted 2026-05-13